CMIS 3153 · ADVANCED DATABASE SYSTEMS

Mid Exam Study Pack · Lesson 4

Query Optimization

Lesson in simple English · Formula cheat box · Answer recipes · Tutorial answers · Past paper questions 2020/21 – 2023/24 with answers

INSIDE THIS PACK

- How to use this pack
- Lesson part 1: Executing queries and measuring their cost
- Lesson part 2: Optimization strategies, query trees and indexing
- Cheat box: rules and formulas for Lesson 4
- How-to-answer recipes
- Tutorial 4: Selectivity and index cost
- Lesson 4 slide exercises
- Past paper questions with answers

★ asked in more than one past paper · (25m) marks in the paper · Yellow boxes = past paper questions · Blue boxes = formulas and key notes · Green boxes = given data

Contents

How to use this pack 4

Quick-find table 4

Lesson part 1: Executing queries and measuring their cost 6

1.1 The three key steps (where optimization sits) 6

1.2 Query execution 6

1.3 The cost of a query 7

1.4 Cost models 7

1.5 Logical vs physical query plans (and optimization) 7

Lesson part 2: Optimization strategies, query trees and indexing 9

2.1 Cost-based optimization (CBO) 9

2.2 Rule-based (heuristic) optimization (RBO) 9

2.3 Query trees 9

2.4 Query graphs 10

2.5 Query trees vs query graphs 10

2.6 Heuristic optimization of query trees 11

2.7 The role of indexing in query optimization 11

Cheat box: rules and formulas for Lesson 4 13

The 12 transformation rules (equivalences) 13

Heuristic optimization: the 6 steps 13

Selectivity and cost formulas 14

How-to-answer recipes 15

Recipe 1: Draw the initial (canonical) query tree from SQL ★ 15

Recipe 2: Optimize the tree with heuristics ★ 15

Recipe 3: Selectivity and index cost calculations (Tutorial 4 style) 16

Recipe 4: "Why might plan B be faster than plan A?" 16

Recipe 5: Theory questions (compare, discuss) 16

Tutorial 4: Selectivity and index cost 17

The SQL used in this tutorial, explained 17

Worked examples (solutions from the tutorial, explained) 17

Exercises (full solutions) 19

Lesson 4 slide exercises 21

(a) Practice questions 21

(b) Essay questions (short answers) 23

Past paper questions with answers 25

A. Optimization steps; logical vs physical 25

B. Cost-based vs rule-based optimization 26

C. Query trees vs query graphs 27

D. Drawing query trees and logical plans 28

E. Heuristic optimization of a query tree ★ 29

F. Indexing, physical plans and performance 32

Questions NOT in the mid scope 34

How to use this pack

Query optimization has come up in **every past paper**, usually as **Question 5**: compare query trees and query graphs, then **draw an initial query tree for an SQL query and optimize it**. The tree questions are worth up to **60 marks** and follow one fixed recipe (Recipe 1 and 2), so practise drawing them.

Tutorial 4 (selectivity and index cost) is in this pack because it is about how the optimizer decides whether to use an index.

In the exam hall:

1. Find the question type in the Quick-find table below.
2. For a query tree question, follow Recipe 1 (initial tree) and Recipe 2 (optimize), naming the rule for each step.
3. Match the length to the marks: a **20–25-mark** part needs **4–6 clear points**, a comparison table, or a full drawing.

Signs used in this pack: ★ = asked in more than one past paper. (20m) = marks in the paper. σ = select (filter rows), π = project (keep columns), ⋈ = join, × = Cartesian product.

Quick-find table

If the question asks about...	Go to
Logical vs physical query optimization (with examples) ★	Lesson part 1 · 2022/23 Q4(b), 2023/24 Q4(b)
The three key steps of query optimization (diagram)	Lesson part 1 · 2021/22 Q7(a)
Cost-based vs rule-based (heuristic) optimization ★	Lesson part 2 · 2021/22 Q7(b), 2022/23 Q5(b)
Query trees vs query graphs, join ordering ★	Lesson part 2 · 2022/23 Q5(c), 2023/24 Q5(b)
Draw a query tree / logical plan from SQL	Recipe 1 · 2020/21 Q2(a), 2022/23 Q5(d)
Optimize a query tree with heuristic rules ★	Recipe 2 · 2021/22 Q7(c), 2023/24 Q5(c)
Transformation rules (cascade of σ, commuting σ with ⋈, ...)	Cheat box
Why physical plan B is faster than plan A	Lesson part 1 (join algorithms) · 2020/21 Q2(b)
Role of indexing in query optimization ★	Lesson part 2 · 2022/23 Q5(a), 2023/24 Q2(a)
Reasons for poor query performance and remedies	2020/21 Q7(a)
Join algorithms (nested loop, hash, sort-merge)	Lesson part 1 · Slide essay 8
Cost of a query, cost models	Lesson part 1

If the question asks about...	Go to
Selectivity, will the index be used? Bitmap?	Recipe 3 · Tutorial 4
Index I/O cost, B-tree height, covering index, hash comparisons	Recipe 3 · Tutorial 4

Lesson part 1: Executing queries and measuring their cost

Query optimization = choosing the **most efficient execution plan** out of the many possible plans for the same query. The **system** does it (not the user). Different plans give the **same result** but can differ hugely in cost.

Two main techniques, usually **combined** in a real optimizer:

1. **Heuristic rules:** reorder the operations in the query tree using rules that work well in most cases (e.g. filter early).
2. **Cost estimation:** estimate the cost of different plans and pick the cheapest.

1.1 The three key steps (where optimization sits)

```
SQL query --> (1) Parser & translator --> relational algebra / initial query tree
                                                    |
                    statistics about the data --> (2) Optimizer --> execution plan
                                                    |
                                 data ------> (3) Evaluation engine --> query result
```

1. **Parsing and translation:** check syntax and names, translate to relational algebra (initial query tree).
2. **Optimization:** generate equivalent plans (heuristics + alternative algorithms), estimate costs with **catalog statistics**, choose the cheapest = the **execution plan**.
3. **Evaluation:** the execution engine runs the plan and returns the result.

1.2 Query execution

An **execution plan** is the sequence of operations the optimizer chose, drawn as a tree. It decides: which **indexes** to use, which **join algorithms**, whether to run in **parallel**, and the **order** of operations.

Scan operations (access paths):

- **Table scan (full scan):** read every row/block in order. Good when many rows match.
- **Index scan:** use an index to reach only matching rows. Good for selective conditions (`WHERE age = 30`, small ranges). A clustered index is fastest for ranges; a non-clustered index needs an extra lookup per row.

Join algorithms:

Algorithm	How it works	Best when
Nested loop join	For each row of the outer table, scan the inner table for matches	Small tables, or the outer input is small
Index nested loop join	For each outer row, **look up** matches using an index on the inner table's join column	Few outer rows + an index on the inner join column

Algorithm	How it works	Best when
Hash join	Build a hash table on the smaller input, then probe it with each row of the other input	Large tables, equality join, no useful index
Sort-merge join	Sort both inputs on the join column, then merge them in one pass	Inputs already sorted (or the output must be sorted)

Execution phases: fetch data (tables/indexes) → apply filters (WHERE) → join → project (SELECT columns) → aggregation and sorting (GROUP BY, ORDER BY) last. GROUP BY and ORDER BY need sorting, which is costly without a sorted index.

1.3 The cost of a query

Cost = the **resources** needed to process the query and return the result:

- **I/O (disk access) cost:** number of blocks read/written — usually the **biggest** cost (table scans vs index scans).
- **CPU cost:** filtering, joining, sorting, aggregating, calculations.
- **Memory cost:** buffers for sorting, grouping, intermediate results. If memory runs out, the DBMS uses disk (external sort), adding I/O.
- **Storage cost:** temporary files for intermediate results.
- **Communication / data transfer cost:** shipping data between sites (distributed and cloud databases).
- **Execution time** (and in the cloud, money per query).

1.4 Cost models

A **cost model** is a mathematical way to **estimate** the resources each plan will use, so the optimizer can compare plans.

- **Simple cost model:** basic rules (index scan cheaper than table scan; nested loops cheaper for small data). Easy but ignores data distribution, so it can be inaccurate.
- **Statistical cost model:** uses **statistics** (number of rows, distinct values, value distribution/histograms). Key parts: **cardinality estimation** (how many rows each step produces) and **selectivity estimation** (what fraction of rows a condition matches).
- **Cascades framework:** applies transformation rules to generate new plans and estimates their costs, using **dynamic programming** to search efficiently.
- **Selectivity (sl)** of a condition = fraction of rows that satisfy it. For an equality on an attribute with d distinct values: **sl = 1/d** and expected matching rows **s = r/d** (for a key: s = 1).

1.5 Logical vs physical query plans (and optimization)

	Logical plan / logical optimization	Physical plan / physical optimization
Answers	**What** to do	**How** to do it
Form	Relational algebra expression / query tree (σ, π, ⋈, γ, sort)	Operator tree with **algorithms and access paths**

	Logical plan / logical optimization	Physical plan / physical optimization
Depends on indexes, storage, hardware?	**No**	**Yes**
Technique	**Rewriting** with equivalence rules / heuristics	**Cost estimation** with statistics
Examples	Push selections down, push projections down, reorder joins, turn σ over × into ⋈, remove redundant operations	Index scan vs full scan; hash join vs nested loop vs sort-merge; in-memory vs external sort; pipelining; parallelism

Logical example: π name (σ dept_name='Sales' (Emp ⋈ Dept)) → π name (Emp ⋈ σ dept_name='Sales' (Dept)). The filter runs before the join, so the join handles far fewer rows.

Physical example (lecture): `SELECT first_name, last_name FROM employees WHERE department = 'Sales' ORDER BY first_name;`

- Logical plan: filter department = 'Sales' → project first_name, last_name → sort by first_name.
- Physical plan: use an **index on department** (if one exists) instead of a full scan; use an **in-memory sort** if the result is small, otherwise an **external merge sort**; if there were joins, choose **hash join** or **nested loops**.

Lesson part 2: Optimization strategies, query trees and indexing

2.1 Cost-based optimization (CBO)

The DBMS **generates several possible plans**, **estimates the cost** of each with a cost model, and **picks the cheapest**.

- **Steps:** (1) **plan generation** (different join orders, predicate pushdown, access paths, join algorithms) → (2) **cost estimation** (I/O, CPU, memory, network) using statistics: table sizes (**cardinality**), **selectivity** of conditions, histograms, available indexes → (3) **plan selection** (lowest estimated cost).
- **Pros:** accurate; adapts to the actual data and hardware.
- **Cons:** optimizing can itself be **expensive** for complex queries (huge number of plans); depends on **accurate, up-to-date statistics**; estimates can be wrong.

2.2 Rule-based (heuristic) optimization (RBO)

Applies a fixed set of **rules** that usually make any query faster, **without computing costs**.

- **Typical rules:** push selections down (filter early); push projections down (drop unneeded columns early); do the **most restrictive** selections and joins first; replace × + σ with ⋈; reorder joins to keep intermediate results small; remove redundant operations; use an index when one exists.
- **Pros:** fast and simple; low optimization overhead; good results quickly.
- **Cons: not guaranteed optimal**; **ignores the real data distribution** (it might use an index on a condition that matches 80% of the rows).

The fundamental difference: heuristic optimization decides using **general rules about the shape of the query**; cost-based optimization decides using **numerical cost estimates from statistics about the actual data**. Real optimizers are **hybrid**: heuristics first (to simplify and cut down the options), then cost-based choice among what remains.

	Heuristic / rule-based	Cost-based
Basis	Fixed rules	Estimated cost
Uses statistics?	No	Yes, essential
Plans compared	One transformed plan	Many alternatives
Optimization time	Low	Higher
Plan quality	Usually good, not guaranteed	Usually better (if statistics are good)

2.3 Query trees

A **query tree** is a tree that represents a **relational algebra expression**.

- **Leaf nodes** = the input **tables** (base relations).

- **Internal nodes** = **operators**: σ (selection), π (projection), ⋈ (join), ×, aggregation (γ), sorting (τ).
- **Root** = the final operation that produces the result.
- Executed **bottom-up**: an internal node runs when its children's results are available. So a query tree fixes an **order of operations**.
- The **canonical (initial) tree** of an SQL query = × of all tables in FROM at the bottom, **one σ** with all WHERE conditions above, **one π** with the SELECT columns at the top. It is correct but never executed (the Cartesian product is huge); the optimizer **transforms** it.
- **Why query trees matter:** an abstract, easy-to-analyse form of the query; breaks a complex query into small operations; the optimizer generates and compares plans by transforming the tree (join reordering, predicate pushdown, projection pushdown, removing redundant operations); different physical strategies (join algorithms, access paths) are attached to its nodes.

2.4 Query graphs

A **query graph** represents a **relational calculus** query.

- **Nodes** = relations (tables), plus **constant** nodes (like 'Stafford').
- **Edges** = **join conditions** (between two relations) and **selection conditions** (between a relation and a constant).
- Attributes to retrieve are written beside each relation.
- It shows **what is connected**, but **no order** of execution. There is **exactly one** graph per query.

Lecture example: *for every project in 'Stafford', list the project number, controlling department, and the department manager's last name, address and birth date.*



2.5 Query trees vs query graphs

	Query tree	Query graph
Represents	Relational algebra expression	Relational calculus / connections between tables
Structure	Hierarchical tree	General graph (can have cycles)

	Query tree	Query graph
Order of operations	**Fixed** (bottom-up)	**Not shown** (neutral)
Per query	**Many** equivalent trees	**Exactly one** graph
Main use	Heuristic rewriting, cost-based plan choice, **execution**	Seeing which tables connect and **which joins are possible**; complex multi-table queries
Preferred by optimizers?	**Yes**, because the plan must state an order	As an aid only

Similarities: both are internal representations made after parsing; both show the relations and the join/selection conditions; both are used in optimization.

How a query graph helps join ordering: its edges show which tables **share a join condition**. Joining two tables with **no edge** between them would create a **Cartesian product**, so the optimizer only considers join orders that follow the edges. Edges to **constants** show which tables are filtered most (e.g. Pname = 'Aquarius' → 1 row), so those are joined **first**. The graph's shape (chain, star, cycle) also limits the sensible orders. The chosen order is then written as a **query tree**.

2.6 Heuristic optimization of query trees

The standard heuristic steps (lecture example: the Aquarius query):

1. **Break up** each selection with AND into a **cascade** of single selections (rule 1).
2. **Push each selection down** the tree as far as possible, directly onto the table(s) it uses (rules 2, 4, 6, 10).
3. **Rearrange the leaves** so the **most restrictive** selections are done first, and so that no Cartesian product is formed between tables that have no join condition (rules 5, 9).
4. **Replace** a Cartesian product followed by a join condition with a **join** (rule 12).
5. **Push projections down**, keeping only the columns needed later (rules 3, 4, 7, 11).
6. Identify subtrees that can run as **pipelines** (no temporary tables).

Why: σ and π make relations **smaller** (fewer rows, fewer columns), while joins and products make them **bigger** (their size multiplies). So shrink first, combine later.

Benefits: faster execution, fewer resources, scales to large data, quick to compute. **Limitations:** not guaranteed optimal; rules are based on typical cases.

2.7 The role of indexing in query optimization

- Indexes are **access paths**. For each table the optimizer chooses **full table scan vs index scan** by estimating cost from **selectivity** (matching rows ÷ total rows) and catalog statistics.
- **Selective conditions** (equality on a key, small ranges): index lookup ≈ height + 1 I/Os vs b blocks for a scan (Tutorial 4 Example 2: **4 I/Os vs 50,000**).
- **Joins:** an index on the inner table's join column enables an **index nested loop join**; sorted indexes help **merge joins**.
- **Avoiding sorts:** reading a B+ tree in order gives ORDER BY, GROUP BY, MIN/MAX for free.

- **Covering indexes** answer the query without touching the table (Tutorial 4 Example 7: **5–6 vs 503 I/Os**).
- The optimizer **does not use an index** when too many rows match (Tutorial 4 Example 1: status = 'pending' matches 25% → full scan) or when a non-clustered index would cause too many random reads.
- **Bitmap index:** good for **low-cardinality** columns (few distinct values, like status or gender); compact and fast to combine with AND/OR.
- **System effects:** (+) faster queries, less I/O and CPU, more users served. (–) slower INSERT/UPDATE/DELETE (every index maintained), extra storage, stale statistics → bad plans.
- **Guidelines:** index columns used in WHERE, JOIN and ORDER BY with good selectivity; clustered index on the main range column; composite/covering indexes for frequent queries; drop unused indexes; keep statistics up to date; check plans with EXPLAIN.

Cheat box: rules and formulas for Lesson 4

The 12 transformation rules (equivalences)

These rewrite a query tree into an **equivalent** one (same result). Name the rule when you use it in an answer.

#	Rule	In symbols (short form)
1	**Cascade of σ:** an AND condition can be split into separate selections	σ c1 ∧ c2 (R) = σ c1 (σ c2 (R))
2	**Commutativity of σ:** selections can be done in any order	σ c1 (σ c2 (R)) = σ c2 (σ c1 (R))
3	**Cascade of π:** in a chain of projections only the last (outermost) matters	π L1 (π L2 (R)) = π L1 (R)
4	**Commuting σ with π:** if the condition uses only projected columns	π L (σ c (R)) = σ c (π L (R))
5	**Commutativity of ⋈ and ×**	R ⋈ S = S ⋈ R ; R × S = S × R
6	**Commuting σ with ⋈ (or ×):** if the condition uses only R's columns, apply it to R first	σ c (R ⋈ S) = σ c (R) ⋈ S ; if c = c1 ∧ c2: σ c1 (R) ⋈ σ c2 (S)
7	**Commuting π with ⋈ (or ×):** project each side to the columns needed (keep join columns)	π L (R ⋈ S) = π L (π L1 (R) ⋈ π L2 (S))
8	**Commutativity of ∪ and ∩** (not −)	R ∪ S = S ∪ R ; R ∩ S = S ∩ R
9	**Associativity of ⋈, ×, ∪, ∩**	(R ⋈ S) ⋈ T = R ⋈ (S ⋈ T)
10	**Commuting σ with set operations (∪, ∩, −)**	σ c (R ∪ S) = σ c (R) ∪ σ c (S)
11	**π commutes with ∪** (but NOT with −)	π L (R ∪ S) = π L (R) ∪ π L (S)
12	**Converting (σ, ×) into ⋈:** a selection that is a join condition over a product = a join	σ c (R × S) = R ⋈ c S

Heuristic optimization: the 6 steps

1. Split AND selections into single selections (rule 1).
2. Push every selection down onto its own table (rules 2, 4, 6, 10).
3. Put the most restrictive selection/join first; never form a × between tables with no join condition (rules 5, 9).
4. Turn × + join condition into ⋈ (rule 12).
5. Push projections down; keep only needed columns + join columns (rules 3, 4, 7, 11).
6. Mark subtrees that can be pipelined.

Memory line: shrink first (σ removes rows, π removes columns), combine later (⋈ multiplies).

Selectivity and cost formulas

$$\text{selectivity} = \frac{\text{rows that match}}{\text{total rows}}$$

$$\text{equality on attribute with } d \text{ distinct values: } sl = \frac{1}{d}$$

$$s = sl \times r = \frac{r}{d}$$

$$(\text{key: } s = 1)$$

$$\text{B-tree point lookup I/O} = \text{height} + 1$$

$$\text{full table scan I/O} = \text{number of pages}$$

Situation	Cost (pages / I/Os)
Point lookup with B-tree index	height + 1 (row fetch)
Full table scan	all pages b
Clustered range scan	(matching rows ÷ rows per page) + a few index pages
Non-clustered range scan	up to 1 random I/O **per matching row**
Covering index	index pages only, no table access
Hash index, uniform	rows per bucket = rows ÷ buckets; comparisons ≈ half a bucket; about 1 bucket I/O
B-tree comparisons	about log2(number of rows)
Composite index (a, b, c) estimate	rows ÷ distinct(a) ÷ distinct(b) ÷ distinct(c)
Index entries per page	floor((page size − overhead) ÷ (key + pointer))

Will the index be used? If selectivity < the threshold given (often 0.10–0.15) → **yes, index**. If above → **full table scan** (too many random reads). Low-cardinality column → a **bitmap index** suits it better.

Useful logs: log2(2,000,000) ≈ 21, log2(4,000,000) ≈ 22. Page sizes: 8 KB = 8,192 bytes, 16 KB = 16,384 bytes.

How-to-answer recipes

Recipe 1: Draw the initial (canonical) query tree from SQL ★

1. **Leaves:** one leaf per table in FROM.
2. **Combine** them with **×** (Cartesian product), pairing two at a time: ((A × B) × C).
3. Above that, **one σ** node with **all** the WHERE conditions joined by ∧.
4. If there is GROUP BY: a **γ** node; HAVING: a σ above it.
5. At the top, **one π** with the SELECT columns. ORDER BY → a sort node τ at the very top.
6. Label every node. (If the question says "query tree" without "initial", you may also draw the joins directly with ⋈, as in Recipe 2.)

```
SELECT A.x, C.z FROM A, B, C WHERE A.id = B.id AND B.k = C.k AND A.y > 5


            pi A.x, C.z
                 |
  sigma A.id=B.id AND B.k=C.k AND A.y>5
                 |
                 x
               /   \
              x     C
            /   \
           A     B
```

Recipe 2: Optimize the tree with heuristics ★

Show each step and **name the rule** (from the Cheat box). Drawing 2–3 intermediate trees earns more marks.

1. **Split** the big σ into single conditions (cascade of σ, rule 1).
2. **Push filters down:** each condition that uses only one table goes directly above that table (rule 6).
3. **Order the tables:** start with the most restrictive filter (a condition on a key or a unique value returns 1 row); never put two tables with **no join condition** next to each other (rules 5, 9).
4. **× + join condition → ⋈** (rule 12).
5. **Push projections down:** above each table keep only the output columns + the join columns (rule 7).
6. Draw the final tree, then list **why it is faster**: no Cartesian products, smaller intermediate results, narrower rows (more per block, less I/O), efficient join algorithms possible, same result.

The same example, optimized:

```
                pi A.x, C.z
                     |
                join B.k = C.k
               /              \
        pi A.x, B.k          pi C.k, C.z
             |                    |
       join A.id = B.id           C
        /            \
  pi A.id, A.x     pi B.id, B.k
       |               |
  sigma A.y > 5        B
       |
       A
```

Recipe 3: Selectivity and index cost calculations (Tutorial 4 style)

1. **Selectivity** = matching rows ÷ total rows. Compare with the threshold → index or full scan.
2. **Low cardinality** (few distinct values) → suggest a **bitmap** index.
3. **B-tree point lookup** = height + 1. Full scan = number of pages. Compare.
4. **Range:** matching rows = rows per day × days. Clustered → pages = matching rows ÷ rows per page. Non-clustered → one random read per row → usually a full scan wins.
5. **Composite index:** divide step by step by the number of distinct values of each column. All columns given with = → **index seek**.
6. **Hash:** rows per bucket = rows ÷ buckets; comparisons ≈ half of that; compare with log2(rows) for a B-tree; hash is useless for ranges.
7. **Index size:** entries per page = (page − overhead) ÷ (key + pointer); leaf pages = rows ÷ entries; go up a level by dividing by entries per page until 1 = root; height = number of levels.
8. **Covering vs non-covering:** covering = index pages only; non-covering = index pages + one row fetch per match.

Recipe 4: "Why might plan B be faster than plan A?"

Look for these differences and explain each one (what A does → why costly → what B does → why cheaper):

- **Index scan with a range on an unclustered index** (many random reads) vs a **sequential file scan**.
- **Index nested loop join** (one index probe per outer row) vs **hash join** (each input read about once).
- **Selection after the join** (bigger join) vs **selection before the join** (predicate pushdown).

Recipe 5: Theory questions (compare, discuss)

Definition of each → a comparison table → one example each → one-line conclusion (for example "real optimizers are hybrid").

Tutorial 4: Selectivity and index cost

> **Exam link:** these calculations support 2020/21 Q5(b), Q6(c) and Q2(b); 2022/23 Q2(b) and Q5(a); 2023/24 Q2(a)(b).

Key ideas:

- **Selectivity** = matching rows ÷ total rows. The tutorial calls a predicate that matches **very few** rows "highly selective" (it is "sensitive"): that is when an index helps most.
- **B-tree point lookup** = height + 1 (the row fetch).
- **Clustered** range scans read consecutive pages; **non-clustered** ones can need a random read **per row**.
- **Bitmap** indexes suit **low-cardinality** columns (few distinct values).
- **Hash** indexes are great for equality, useless for ranges.

The SQL used in this tutorial, explained

SQL	Meaning
`SELECT * FROM Orders WHERE status = 'pending';`	All columns of the orders whose status is exactly 'pending' (an **equality** condition)
`WHERE created_at BETWEEN '2025-01-01' AND '2025-01-03'`	Rows with a date from 1 Jan to 3 Jan **inclusive** (a **range** condition, 3 days)
`WHERE country = 'UK' AND city = 'London' AND age = 30`	All three equalities must be true (matches a **composite index** (country, city, age) exactly)
`SELECT last_name, first_name FROM Employees WHERE last_name = 'Brown';`	Asks only for columns that are **in the index** (last_name, first_name) → **covering**
`SELECT * FROM employees WHERE last_name = 'Brown';`	Asks for **all** columns → the table rows must be fetched → **non-covering**
`CREATE INDEX idx_active ON users(last_login) WHERE active = true;`	A **partial index**: it only contains rows where active = true
`WHERE last_login > NOW() - INTERVAL '7 days'`	Logged in during the last 7 days (NOW() = the current date and time)

Worked examples (solutions from the tutorial, explained)

Example 1: Selectivity. Orders has 1,000,000 rows. status: shipped 600,000, pending 250,000, cancelled 150,000. Query: `WHERE status = 'pending'`.

1. Selectivity = 250,000 ÷ 1,000,000 = **0.25 (25%)**.
2. Threshold 0.10: 0.25 > 0.10 → the B-tree index is **not used**; a full scan is cheaper.
3. A **bitmap index** is better: status has only 5 values (low cardinality), bitmaps compress well and filter very fast, especially combined with other conditions.

Example 2: B-tree I/O. Height 3, each node = 1 I/O, row fetch = 1 I/O, table = 50,000 pages. Query by email.

1. Index: root → level 1 → leaf = 3, + 1 row fetch = **4 I/Os**.
2. Full scan = **50,000 I/Os**.
3. The index is massively more efficient.

Example 3: Range scan vs full scan. 5,000,000 rows, clustered B-tree on created_at, 100,000 rows per day, 3-day range.

1. Matching rows = 3 × 100,000 = **300,000**.
2. Selectivity = 300,000 ÷ 5,000,000 = **6%** → selective enough → the index **is used** (especially because it is clustered).
3. If **non-clustered:** 300,000 **random** reads → too expensive → the optimizer would do a **full table scan**.

Example 4: Composite index (country, city, age). 10,000,000 rows; 50 countries; about 200 cities per country; about 80 ages.

1. After country = 'UK': 10,000,000 ÷ 50 = **200,000**.
2. After city = 'London': 200,000 ÷ 200 = **1,000**.
3. After age = 30: 1,000 ÷ 80 = 12.5 ≈ **13 rows**.
4. All index columns are given with = → **index seek**.

Example 5: Hash index. 2,000,000 rows, 1,000 buckets, uniform.

1. Rows per bucket = 2,000,000 ÷ 1,000 = **2,000**.
2. Comparisons ≈ half a bucket ≈ **1,000**.
3. B-tree ≈ log2(2,000,000) ≈ **21** comparisons. Hash ≈ 1,000 comparisons but **fewer I/Os**, so hash is faster for exact equality; it is useless for ranges, ordering and prefix matching.

Example 6: Index size. Key 20 bytes, pointer 8 bytes, node overhead 100 bytes, page 8 KB = 8,192 bytes, 2,000,000 rows.

1. Usable space = 8,192 − 100 = 8,092. Entry = 20 + 8 = 28 bytes. Entries per page = 8,092 ÷ 28 ≈ **289**.
2. Leaf pages = 2,000,000 ÷ 289 ≈ **6,920**.
3. Level above the leaves: 6,920 ÷ 289 ≈ 23.9 → **24** nodes. Next level: 24 ÷ 289 < 1 → **1** root. So **height = 3** (root → internal → leaf). (The tutorial prints "24/280"; it should be 24/289. Same answer.)
4. Point lookup ≈ 3 + 1 = **4 I/Os**.

Example 7: Covering vs non-covering. Index (last_name, first_name); 500,000 rows; about 500 rows per last name.

1. Index entries scanned = **500**.
2. Query A (covering): about 3 I/Os to find the start + 2–3 leaf pages ≈ **5–6 I/Os**. Query B (non-covering): the same + **500 random row lookups** ≈ **503 I/Os**.
3. Covering is efficient because **no table access** is needed: all the data is in the index.

Exercises (full solutions)

The tutorial gives no answers for these. They use the same method as the examples; where data is missing, the assumption is stated.

Exercise 1: Partial index. 8,000,000 rows, 2,000,000 active. `CREATE INDEX idx_active ON users(last_login) WHERE active = true;` Query: `SELECT * FROM users WHERE active = true AND last_login > NOW() - INTERVAL '7 days';`

- **(i) Index selectivity:** the partial index holds only active rows: 2,000,000 ÷ 8,000,000 = **0.25** of the table. The query asks for active = true, which matches the index's own condition, so only these 2,000,000 entries are searched; the last_login range narrows it further.
- **(ii) Is the index used? Yes, most likely.** The query's condition fits the index condition, the index is 4× smaller than a full index, and "last 7 days" usually matches only a small share of active users (for example, if 5% logged in this week: 100,000 rows = 1.25% of the table). (Assumption: the number of recent logins is not given; if most active users logged in this week, about 25% of the table would match and a full scan could win.)

Exercise 2: Membership level. 5,000,000 rows: Gold 2,000,000; Silver 1,500,000; Bronze 1,200,000; Other 300,000. Query: `SELECT * FROM customers WHERE membership_level = 'Silver';`

1. Selectivity = 1,500,000 ÷ 5,000,000 = **0.30**.
2. Rule "use index only if selectivity < 0.15": 0.30 > 0.15 → **index NOT used** (full scan).
3. **Bitmap index: yes, it helps.** Only 4 distinct values (low cardinality), so bitmaps are small and fast to combine with AND/OR. But this query alone returns 30% of the rows, so the gain is limited; bitmaps shine when combined with other filters.

Exercise 3: Clustered range. Clustered index on order_id. Query: `SELECT * FROM orders WHERE order_id BETWEEN 100000 AND 150000;` → 50,000 consecutive rows, 200 rows per page.

- **(i)** Pages scanned = 50,000 ÷ 200 = **250 pages** (+ about 3 index pages to find the start).
- **(ii) Non-clustered:** the rows are scattered → up to **50,000** random page reads (+ index leaf pages) = about **200 times more**. The optimizer would probably choose a full scan instead.

Exercise 4: Hash index load. 200 buckets, 640 rows, load factor 3.2.

- **(i)** Average rows per bucket = 640 ÷ 200 = **3.2** (this is the load factor).
- **(ii) Probability of overflow** depends on the **bucket capacity**, which is not given. With uniform hashing, the number of rows in a bucket follows a Poisson distribution with mean 3.2. If a bucket holds **4** rows: P(more than 4) ≈ **0.22** (about 22% of buckets overflow). If it holds 5: ≈ 0.11. If it holds 3: ≈ 0.40. (Assumption: capacity not stated.)

Exercise 5: B-tree point lookup. Height 4; each index page = 1 I/O; row = 1 I/O; table = 300,000 pages. Query: `SELECT * FROM employees WHERE ssn = '123-45-6789';`

- **(i)** Index: 4 + 1 = **5 I/Os**.
- **(ii)** Full scan: **300,000 I/Os**.
- **(iii)** The optimizer chooses the **index** (ssn is unique, so it is extremely selective).

Exercise 6: Sensor logs. 20,000,000 rows; clustered index on timestamp; about 150,000 rows per day. Query: `SELECT * FROM sensor_logs WHERE timestamp BETWEEN '2025-04-01' AND '2025-04-02';`

- **(iv) Matching rows** ≈ 2 days × 150,000 = **300,000** (both days counted, as in Example 3).
- **(v) Selectivity** = 300,000 ÷ 20,000,000 = **0.015 (1.5%)**.
- **(vi) Clustered range scan chosen? Yes:** it reads only the contiguous pages for those 2 days.
- **(vii) If non-clustered:** 300,000 random row fetches. Even at 1.5% that is a lot of random I/O, so many optimizers would choose a **full scan**. The clustered case is far better.

Exercise 7: Composite index (state, city, postal_code). 16,000,000 rows; 40 states; 100 cities per state; 50 postal codes per city. Query: `WHERE state = 'CA' AND city = 'San Diego' AND postal_code = '92101'`.

- **(i)** After state: 16,000,000 ÷ 40 = **400,000**.
- **(ii)** After city: 400,000 ÷ 100 = **4,000**.
- **(iii)** After postal code: 4,000 ÷ 50 = **80**.
- **(iv) Yes, an index seek:** all three columns are given with =, in index order.

Exercise 8: Hash on api_tokens. 4,000,000 rows; 2,000 buckets; uniform. Query: `WHERE token = 'abc123'`.

- **(i)** Rows per bucket = 4,000,000 ÷ 2,000 = **2,000**.
- **(ii)** Comparisons ≈ half a bucket ≈ **1,000**.
- **(iii)** B-tree ≈ log2(4,000,000) ≈ **22** comparisons. The hash needs more comparisons inside the bucket but usually **fewer I/Os** (one bucket read), so for equality it is faster in I/O terms; the B-tree also supports ranges.

Exercise 9: Index sizing. Page 16 KB = 16,384 bytes; overhead 120 bytes; key 32 bytes; pointer 8 bytes; 8,000,000 rows.

- **(i)** Entries per page = floor((16,384 − 120) ÷ (32 + 8)) = floor(16,264 ÷ 40) = floor(406.6) = **406**.
- **(ii)** Leaf pages = ceil(8,000,000 ÷ 406) = **19,705**.
- **(iii)** Level 2 = ceil(19,705 ÷ 406) = **49** nodes; level 3 = ceil(49 ÷ 406) = **1** root → **height 3**.
- **(iv)** Point lookup = 3 + 1 = **4 I/Os**.

Exercise 10: Covering vs non-covering. Index (category, price); 1,200,000 items; 8,000 with category = 'Electronics'; 200 entries per page. Query A: `SELECT category, price FROM items WHERE category = 'Electronics';` Query B: `SELECT * FROM items WHERE category = 'Electronics';`

- **(i)** Index entries scanned = **8,000**.
- **(ii)** Query A (covering): about 3 I/Os to find the start + 8,000 ÷ 200 = 40 leaf pages ≈ **43 page reads**.
- **(iii)** Query B (non-covering): the same 43 + up to **8,000** row fetches ≈ **8,043** (fewer if matching rows share pages).
- **(iv) Query A is far more efficient:** the index holds every column it needs, so the table is never read.

Lesson 4 slide exercises

(a) Practice questions

Q1. Construct a query tree for the logical plan of: `SELECT name, age FROM employees WHERE age > 30 ORDER BY name;`

Read it bottom-up: take employees → keep rows with age > 30 → keep name and age → sort by name.

```
tau name                (sort, ORDER BY name)
    |
pi name, age            (SELECT name, age)
    |
sigma age > 30          (WHERE age > 30)
    |
employees
```

Q2. employees / departments join with salary > 50000. This is the same as **2022/23 Q5(d)**; the full answer is in the past papers section.

Q3. `SELECT name FROM employees WHERE age > 30 AND department_id IN (SELECT department_id FROM departments WHERE department_name = 'HR');`

I. Query tree. The IN-subquery becomes a join (or semi-join) on department_id with the HR departments.

```
          pi name
             |
      sigma age > 30
             |
join employees.department_id = departments.department_id
     /                          \
employees                pi department_id
                                 |
                    sigma department_name = 'HR'
                                 |
                            departments
```

II. Predicate pushdown. Move `age > 30` down onto employees (it uses only employees' columns; rule 6):

```
         pi name
            |
   join employees.department_id = departments.department_id
        /                          \
  sigma age > 30           pi department_id
        |                          |
   employees               sigma department_name = 'HR'
                                   |
                              departments
```

III. Effect. Only employees older than 30 enter the join, so the join input and the intermediate result are much smaller → fewer rows to compare, less memory, less I/O. The result is the same.

IV. Change to the execution plan. employees is read with the filter applied during the scan (or through an index on age if one exists and is selective); the HR department IDs (a tiny set) are found first; then a **hash join / semi-join** probes the small HR set with each filtered employee (or an **index nested loop** uses an index on employees.department_id). The plan does far less work than joining all employees first.

Q4. `SELECT e.name, d.department_name FROM employees e JOIN departments d ON e.department_id = d.department_id JOIN locations l ON d.location_id = l.location_id WHERE e.salary > 50000;`

Initial query tree:

```
              pi e.name, d.department_name
                          |
                sigma e.salary > 50000
                          |
             join d.location_id = l.location_id
               /                            \
   join e.department_id = d.department_id   locations l
        /                \
   employees e        departments d
```

I. Most likely join order: as written, (employees ⋈ departments) ⋈ locations, with the salary filter pushed down onto employees.

II–V. Using the indexes on employees.department_id and departments.location_id:

- The index on **departments.location_id** lets the DBMS start from locations and, for each location, **look up** its departments (index nested loop join).
- The index on **employees.department_id** lets it, for each department, **look up** its employees (index nested loop join) instead of scanning all employees.
- The salary filter is applied to each employee row as it is fetched (or first, if an index on salary were selective).
- Note: locations gives **no output columns and no filter**. If d.location_id is a NOT NULL foreign key, the join with locations changes nothing and can be **removed** (join elimination).

```
            pi e.name, d.department_name
                        |
    join e.department_id = d.department_id  (index NL on employees.department_id)
          /                          \
join d.location_id = l.location_id     sigma e.salary > 50000
(index nested loop,                          |
 index on departments.location_id)       employees e
     /              \
locations l     departments d
```

(b) Essay questions (short answers)

1. Role of indexing in query optimization. Indexes give the optimizer cheaper **access paths**: a few I/Os instead of a full scan for selective conditions; index nested loop joins; sorted output without sorting; covering indexes avoid the table. **Downsides:** slower inserts/updates/deletes, extra space. **Which columns:** those in WHERE, JOIN and ORDER BY with good selectivity. **Full scan despite an index:** check selectivity and statistics; make the condition index-friendly (no function on the column), update statistics, use a covering or composite index.

2. Execution plans. An **execution plan** shows the steps, access paths, join algorithms and estimated rows/costs the optimizer chose (seen with EXPLAIN). **Bottlenecks:** full scans on big tables, huge row estimates, expensive sorts, nested loops over large inputs, estimates very different from actual rows. **Use:** add or change indexes, rewrite joins/subqueries, update statistics, then compare plans again.

3. Joins vs subqueries. A **join** lets the optimizer choose the join order and algorithm; a **correlated subquery** may run once per outer row. Subqueries are fine (and readable) for small lists with IN/EXISTS. Many optimizers rewrite subqueries into joins/semi-joins. Example: `WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Colombo')` → `JOIN departments d ON e.dept_id = d.dept_id WHERE d.location = 'Colombo'` lets a hash join run once instead of repeated lookups.

4. Data distribution. Skewed data (a few values very common) breaks the uniform assumption: an index good for a rare value is bad for a common one; joins on skewed keys create very large partitions. **Histograms** record how values are really spread, so the optimizer estimates selectivity per value correctly and picks index vs scan and the join order better.

5. Query refactoring. Rewriting a query (same result) so it runs faster. Patterns: `SELECT *` → only needed columns; remove redundant joins and unnecessary ORDER BY / DISTINCT; replace correlated subqueries with joins; avoid functions on indexed columns. For GROUP BY / ORDER BY: filter first, use an index that matches the order, group on fewer columns. Balance: keep it readable (clear aliases, comments); optimize only where it matters.

6. SQL → query tree. Parse → tables in FROM become **leaves** → joins/× combine them → WHERE becomes σ → GROUP BY becomes γ, HAVING a σ above it → SELECT becomes π at the top; UNION becomes a ∪ node with two subtrees; ORDER BY a sort node. **Challenges:** nested and correlated subqueries, many joins (huge number of possible trees), outer joins, NULLs, views.

7. Query caching. The DBMS stores the **results (or plans)** of queries; an identical later query is answered from the cache without running again. Best for **frequent, repeated, read-mostly** queries

(e.g. a product catalogue page, daily sales totals on a dashboard). **Problems:** cached results become **stale** when data changes, so the cache must be **invalidated**; memory use; little benefit when data changes often.

8. Join algorithms. Nested loop: for each outer row scan the inner; good for small inputs. **Index nested loop:** probe an index on the inner join column; good for few outer rows. **Hash join:** build a hash table on the smaller input and probe; best for large equality joins without indexes. **Sort-merge:** sort both inputs and merge; good when inputs are already sorted. The optimizer chooses from **input sizes, indexes, sort order, memory** and the join condition. Example: joining two 1-million-row tables on an unindexed column → a hash join reads each table about once, while a nested loop would compare 10^{12} pairs. Many joins: filter early, join the most selective tables first, avoid Cartesian products, keep statistics current.

9. Query tree and its role. A query tree represents the query as RA operators (leaves = tables, internal nodes = operators, root = result); it fixes the **bottom-up order** of execution. The logical tree is transformed (rules) and then annotated with algorithms to become the **physical plan**. It helps the DBMS choose the best strategy by giving a structure to rewrite and to cost.

10. Query trees vs query graphs. See Lesson part 2, section 2.5 (comparison table) and past paper 2022/23 Q5(c). A query graph is more useful for seeing **which tables connect** (which joins are possible, avoiding Cartesian products) and for complex multi-table or multi-database queries; query trees are needed to state the execution order.

11. Query trees and join order. Each tree shape fixes a join order (left-deep trees join one table at a time). The optimizer generates different tree shapes, estimates each one's cost (intermediate result sizes from selectivity), and keeps the cheapest, often with **dynamic programming**. Factors: table sizes, selectivity of filters, available indexes, join type and algorithm, data distribution. Example: in A ⋈ B ⋈ C, if a filter makes C tiny, join C first.

12. Cost-based optimization and query trees. CBO builds **alternative query trees**, attaches algorithms to each node, estimates each node's **I/O and CPU cost** from statistics (row counts, selectivity), adds them up, and chooses the tree with the **lowest total**. Example: for a 3-table join, the tree that applies the selective filter and joins the smallest results first can cost thousands of I/Os instead of millions.

Past paper questions with answers

Grouped by topic. ★ = asked in more than one paper. Query tree drawings use plain words (pi = π, sigma = σ, join = ⋈, x = ×) so they print clearly; use the symbols when you draw them in the exam.

A. Optimization steps; logical vs physical

2021/22 Q7(a) (20m). Explain the THREE key steps of query optimisation using a diagram.

```
SQL query --> (1) Parser & translator --> relational algebra / initial query tree
                                                  |
                statistics about the data --> (2) Optimizer --> execution plan
                                                  |
                                data ------> (3) Evaluation engine --> query result
```

1. **Parsing and translation:** the query is scanned and parsed, names are checked against the catalog, and it is translated into an internal **relational algebra** form (the initial query tree).
2. **Optimization:** equivalent expressions and alternative algorithms/access paths are generated (heuristic rewriting + cost estimation using **catalog statistics**); the plan with the **lowest estimated cost** becomes the **execution plan**.
3. **Evaluation:** the **query execution engine** runs the plan and returns the result.

2022/23 Q4(b) (25m) and 2023/24 Q4(b) (25m) ★ Explain the difference between logical and physical query optimization, with examples of each.

- **Logical optimization** works on the **logical plan**: a relational algebra expression / query tree that says **what** operations are needed (σ, π, ⋈, γ, sort), **without** saying how; it does not depend on indexes, storage or hardware. It **rewrites** the expression into an equivalent, cheaper one using transformation rules: **predicate pushdown**, **projection pushdown**, **join reordering**, turning σ over × into ⋈, removing redundant operations.
  - *Example:* π name (σ dept_name='Sales' (Emp ⋈ Dept)) → π name (Emp ⋈ σ dept_name='Sales' (Dept)). The filter runs before the join, so far fewer rows are joined.
- **Physical optimization** turns the logical plan into a **physical plan** that says **how** each operation runs: **access paths** (full scan vs index scan), **join algorithms** (nested loop, index nested loop, hash, sort-merge), **sort method** (in-memory vs external merge sort), pipelining, parallelism. Chosen by **cost estimation** with statistics.
  - *Example (lecture):* `SELECT first_name, last_name FROM employees WHERE department = 'Sales' ORDER BY first_name;` → use an **index scan on department** instead of a full scan; an **in-memory sort** if the result is small, otherwise an **external merge sort**; for joins choose **hash join** or **nested loops**.

	Logical	Physical
Question	What to do	How to do it
Form	RA expression / query tree	Operator tree with algorithms and access paths
Depends on indexes and storage?	No	Yes
Technique	Equivalence rules, heuristics	Cost models, statistics
Example	Push σ below ⋈	Index scan; hash join

B. Cost-based vs rule-based optimization

> **2021/22 Q7(b) (20m) and 2022/23 Q5(b) (20m) ★** Describe cost-based and rule-based query optimisation. / Discuss cost-based and heuristic-based optimization; what is the fundamental difference?

- **Cost-based optimization (CBO):** generates **several alternative plans** (join orders, access paths, join algorithms), **estimates each one's cost** with a cost model (CPU, disk I/O, memory, network) using **statistics** (table sizes, **selectivity**, histograms, indexes), and **chooses the cheapest**. Steps: plan generation → cost estimation → plan selection. (+) accurate, adapts to the data and hardware. (−) optimization can be expensive for complex queries; needs accurate, up-to-date statistics.
- **Rule-based / heuristic optimization (RBO):** applies fixed **transformation rules** that usually help: push selections and projections down, do the most restrictive operations first, replace × + σ with ⋈, avoid unnecessary sorting. (+) fast, simple, low overhead. (−) static; **ignores the actual data**, so it can choose badly (e.g. using an index on a condition that matches 80% of rows).
- **Fundamental difference:** heuristic optimization decides by **general rules about the form of the query**; cost-based optimization decides by **numerical cost estimates from statistics about the actual data**. Heuristics do not compare plans quantitatively; CBO does.
- **In practice:** a **hybrid**: heuristics first (simplify and cut down the choices), then a cost-based choice among the rest.

	Heuristic / rule-based	Cost-based
Basis	Fixed rules	Estimated cost
Statistics	Not used	Essential
Plans compared	One transformed plan	Many alternatives
Optimization time	Low	Higher
Plan quality	Usually good, not guaranteed	Usually better (good statistics)

C. Query trees vs query graphs

> **2022/23 Q5(c) (20m) and 2023/24 Q5(b) (20m) ★** Compare and contrast query trees with query graphs, highlighting how query graphs assist in join ordering and query optimization.

- **Query tree:** represents a **relational algebra expression**. Leaves = input tables; internal nodes = operators (σ, π, ⋈, ×, γ, sort); root = the result. Executed **bottom-up**, so it fixes the **order** of operations.
- **Query graph:** represents a **relational calculus** query. Nodes = tables (and **constants**); edges = **join conditions** (table–table) and **selection conditions** (table–constant); output columns written beside the tables. Shows **what is connected**, but **no order**.

	Query tree	Query graph
Represents	RA expression	Relational calculus / connections
Structure	Hierarchical tree	General graph (may have cycles)
Order of operations	Specified	Not specified
Per query	Many equivalent trees	Exactly one graph
Used for	Heuristic rewriting, cost-based choice, execution	Seeing which joins are possible
Preferred by optimizers	Yes (the plan must state an order)	As an aid

Similarities: both are internal forms produced after parsing; both show the tables and the join/selection conditions; both are used in optimization.

How query graphs help join ordering:

- Edges show **which tables share a join condition**. Joining two tables with **no edge** would make a **Cartesian product**, so the optimizer only considers orders that follow the edges.
- The graph's **shape** (chain, star, cycle) limits the sensible orders (e.g. dynamic programming over connected parts).
- **Edges to constants** show which tables are reduced most (e.g. Pname = 'Aquarius' → 1 row); those are joined **first**.
- Because it has no order, the optimizer can explore orders freely; the chosen one is then written as a **query tree**.
- *Example (lecture, Stafford query):* P —(P.Dnum = D.Dnumber)— D —(D.Mgr_ssn = E.Ssn)— E, plus P —(Plocation = 'Stafford')— constant. The chain shows P must be joined to D before E (no P–E edge), and the constant says start from P.

D. Drawing query trees and logical plans

2022/23 Q5(d) (20m + 10m + 10m) (= lecture practice Q2). `SELECT e.name, d.department_name FROM employees e INNER JOIN departments d ON e.department_id = d.department_id WHERE e.salary > 50000;` (i) Construct a query tree. (ii) Show the join as a separate node and explain whether the filter salary > 50000 is applied before or after the join. (iii) Give the expected order of operations.

(i) Query tree — initial form (direct translation):

```
       pi e.name, d.department_name
                  |
        sigma e.salary > 50000
                  |
   join e.department_id = d.department_id
           /              \
     employees e      departments d
```

Optimized form (filter pushed down):

```
       pi e.name, d.department_name
                  |
   join e.department_id = d.department_id
           /              \
sigma e.salary > 50000   departments d
         |
     employees e
```

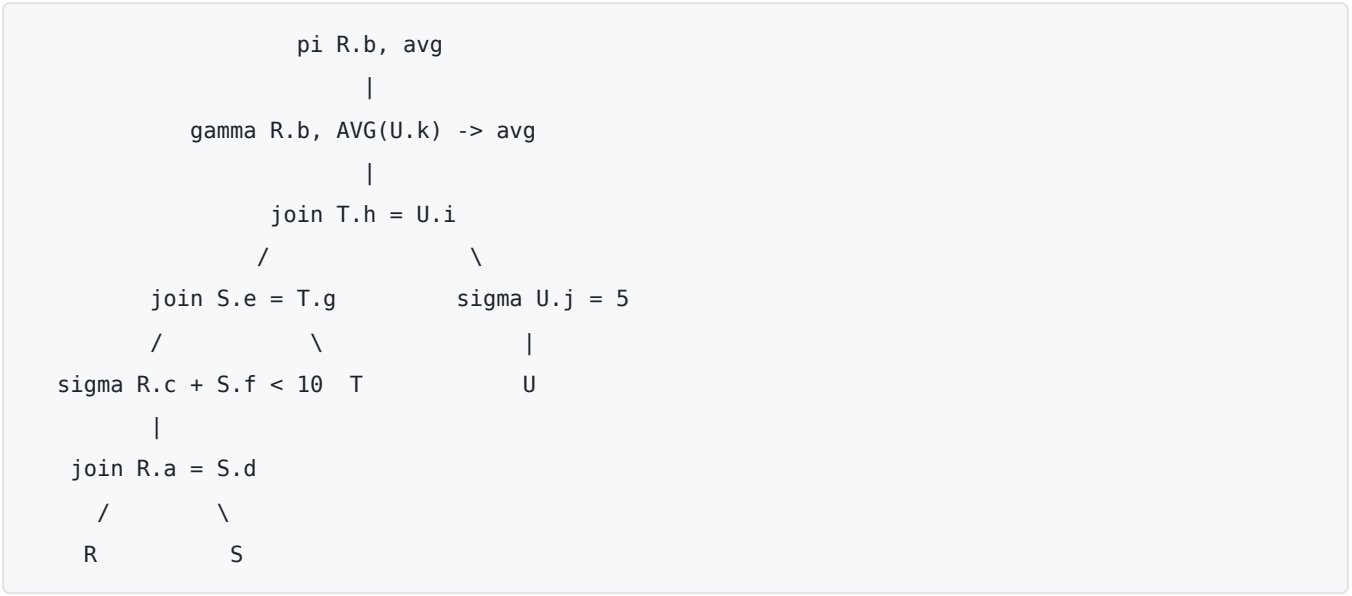
(ii) The join is the separate node ⋈ e.department_id = d.department_id with the two tables as children. The condition salary > 50000 uses **only employees' columns**, so by the rule **commuting σ with ⋈** (σ c (E ⋈ D) = σ c (E) ⋈ D) it can be applied **before the join** (**predicate pushdown**). Both placements give the **same result**, but **before** is better: fewer employee rows enter the join, so the intermediate result is smaller (less I/O and CPU). Applying it after means joining every employee first, then throwing most rows away.

(iii) Order of operations (optimized):

1. Scan **employees** (or use an index on salary if it is selective).
2. Apply **σ salary > 50000**.
3. Scan **departments**.
4. **Join** on department_id (hash join, or index nested loop using an index on departments.department_id).
5. **Project** e.name, d.department_name and return the result.

2020/21 Q2(a) (40m). R(a, b, c), S(d, e, f), T(g, h), U(i, j, k). Draw a logical plan (any correct plan) for:

```
SELECT R.b, avg(U.k) AS avg
FROM R, S, T, U
WHERE R.a = S.d AND S.e = T.g AND T.h = U.i
  AND U.j = 5 AND (R.c + S.f) < 10
GROUP BY R.b;
```



- The condition R.c + S.f < 10 uses **both R and S**, so it can only be applied **after R ⋈ S**.
- U.j = 5 uses only U, so it goes directly on U.
- Grouping (γ) comes **after** all WHERE conditions; π is at the top.
- A simpler (canonical) plan is also correct: π R.b, avg (γ R.b, AVG(U.k)→avg (σ all conditions (R × S × T × U))).

E. Heuristic optimization of a query tree ★

2021/22 Q7(c) (60m). The initial query tree is π Lname (σ Pname='Aquarius' ∧ Pnumber=Pno ∧ Essn=Ssn ∧ Bdate>'1957-12-31' ((EMPLOYEE × WORKS_ON) × PROJECT)). Sample data: EMPLOYEE(SSN, Lname, Bdate) = (12329, Rogers, 1957-12-31), (13453, Ahmed, 1978-03-04), (15463, Bohm, 1956-06-03); WORKS_ON(ESSN, Pnumber, HrsWorked) = (12329, 49, 450), (13453, 43, 200), (15463, 46, 19); PROJECT(Pno, Pname, Pstartdate) = (43, Aquarius, 2021-02-03), (44, Markets, 2020-12-11), (46, Hospitality, 2020-12-24). (i) Write the equivalent SQL. (ii) Explain why the tree is inefficient. (iii) Describe how to transform it to run efficiently.

(i) SQL:

```
SELECT Lname
FROM EMPLOYEE, WORKS_ON, PROJECT
```

```
WHERE Pname = 'Aquarius' AND Pnumber = Pno AND Essn = Ssn
  AND Bdate > '1957-12-31';
```

(Last names of employees born after 1957 who work on the Aquarius project. With the sample data the answer is **Ahmed**.)

(ii) Why the tree is inefficient:

- It is the **canonical tree**: it first builds the **Cartesian product of all three tables**. Even the tiny sample gives 3 × 3 × 3 = 27 rows; real sizes (1,000 × 10,000 × 100) give **10^9 rows**, almost all useless.
- **All the conditions are applied only at the end**, after this huge intermediate result is built, although only **one PROJECT row** (Aquarius) and only employees born after 1957 are needed.
- **All columns** are carried through every step, although only Lname and the join columns are needed.
- The join conditions are written as a selection over ×, so no efficient join algorithm can be used.
- Result: enormous intermediate results → high I/O, memory and CPU cost.

(iii) Transformations, step by step (with the rule used):

1. **Split** the big selection into single selections (cascade of σ, rule 1).
2. **Push selections down:** σ Bdate > '1957-12-31' onto EMPLOYEE, σ Pname = 'Aquarius' onto PROJECT; put each join condition directly above the × it belongs to (commuting σ with ×, rule 6).
3. **Most restrictive first / reorder the leaves:** σ on PROJECT returns **one row** (Pname is unique), so join PROJECT with WORKS_ON first, then EMPLOYEE. This also avoids EMPLOYEE × PROJECT, which has no join condition (commutativity and associativity, rules 5 and 9).

```
After steps 1-3:
                    pi Lname
                       |
                 sigma Essn = Ssn
                       |
                       x
                /            \
     sigma Pnumber = Pno     sigma Bdate > '1957-12-31'
              |                        |
              x                    EMPLOYEE
           /     \
  sigma Pname =    WORKS_ON
   'Aquarius'
       |
    PROJECT
```

4. **Replace × + join condition with ⋈:** σ Pnumber=Pno (... × ...) → ⋈ Pnumber=Pno, and σ Essn=Ssn (... × ...) → ⋈ Essn=Ssn (rule 12).
5. **Push projections down:** keep only Pno from PROJECT; Essn and Pnumber from WORKS_ON; Ssn and Lname from EMPLOYEE; only Essn after the first join (rule 7).

```
Final optimized tree:
                          pi Lname
                              |
                       join Essn = Ssn
                    /                   \
              pi Essn                 pi Ssn, Lname
                 |                          |
        join Pnumber = Pno      sigma Bdate > '1957-12-31'
          /            \                    |
      pi Pno      pi Essn, Pnumber      EMPLOYEE
        |               |
  sigma Pname =      WORKS_ON
   'Aquarius'
        |
     PROJECT
```

With the sample data: Aquarius → Pno 43 → WORKS_ON (13453, 43) → EMPLOYEE 13453 Ahmed (born 1978) → **Ahmed**. Every intermediate result has **1 row** instead of 27.

> **2023/24 Q5(c) (20m + 20m + 20m).** For the query below: (i) draw the initial query tree; (ii) using heuristic rules, convert it into an optimized tree; (iii) explain how the optimized tree improves efficiency.

```sql
SELECT E.Name, D.Dept_Name
FROM Employee E, Department D, Works W
WHERE E.Emp_ID = W.Emp_ID AND W.Dept_ID = D.Dept_ID
  AND D.Location = 'Colombo' AND E.Salary > 80000;
```

(i) Initial (canonical) tree: × of all tables in FROM order, one σ with all conditions, one π.

```
   pi E.Name, D.Dept_Name
            |
   sigma E.Emp_ID = W.Emp_ID AND W.Dept_ID = D.Dept_ID
         AND D.Location = 'Colombo' AND E.Salary > 80000
            |
            x
          /   \
         x     W
       /   \
      E     D
```

(ii) Optimized tree. Steps: split the σ (rule 1) → push E.Salary > 80000 onto E and D.Location = 'Colombo' onto D (rule 6) → reorder the leaves to E – W – D so every × has a join condition (E and D share no condition) (rules 5, 9) → turn σ + × into ⋈ (rule 12) → push projections down (rule 7).



(Also correct: start with σ Location = 'Colombo' (D) ⋈ W if Colombo departments are more selective than high salaries.)

(iii) Why the optimized tree is more efficient:

1. **No Cartesian products:** the initial tree builds |E| × |D| × |W| rows; the optimized tree joins only matching rows.
2. **Selections first (predicate pushdown):** only employees earning > 80,000 and only Colombo departments reach the joins → much smaller intermediate results.
3. **Projections early:** narrower rows → more rows per block → fewer disk I/Os and less memory.
4. **Join order follows the join conditions**, with the reduced inputs joined first (most restrictive first).
5. Smaller inputs allow efficient **join algorithms** (in-memory hash join, index nested loop) and **pipelining**. Overall: lower I/O, CPU and memory cost, and the **same result**.

F. Indexing, physical plans and performance

> **2022/23 Q5(a) (20m) and 2023/24 Q2(a) (25m) ★** Explain the role of indexing in query optimization. / Discuss how indexing influences query optimization and system performance.

Role in optimization:

- Indexes are **access paths**. For each table the optimizer chooses **full table scan vs index scan** by estimating cost from **selectivity** (matching ÷ total rows) and catalog statistics (index height, distinct values, clustering).
- **Selective conditions** (equality on keys, small ranges): index lookup ≈ height + 1 I/Os vs b blocks for a scan (Tutorial 4 Ex. 2: **4 I/Os vs 50,000**).
- **Joins:** an index on the inner table's join column allows an **index nested loop join**; sorted indexes allow **merge joins**.

- **Avoiding sorts:** reading a B+ tree in order gives ORDER BY, GROUP BY, MIN/MAX without sorting.
- **Covering indexes** answer queries without reading the table (Tutorial 4 Ex. 7: **5–6 vs 503 I/Os**).
- **Unique indexes** tell the optimizer a condition returns at most 1 row, which helps choose the join order.
- The optimizer **does not use an index** when too many rows match (status = 'pending' = 25% → full scan; a bitmap index suits low cardinality) or when a non-clustered index would cause too many random reads.

Effect on system performance:

- (+) Faster queries, less I/O and CPU per query → better response time and throughput, more users at once, less contention.
- (−) **Write overhead:** every INSERT/UPDATE/DELETE maintains every index (splits, merges); extra **storage**; too many indexes enlarge the optimizer's choices and maintenance work; stale statistics → bad plans.
- **Guidelines:** index columns used in WHERE / JOIN / ORDER BY with good selectivity; clustered index on the main range column; composite/covering indexes for frequent queries; drop unused indexes; keep statistics current; check with EXPLAIN.

> **2020/21 Q2(b) (60m).** Two physical plans for the same query. Give two reasons (with explanation) why plan B may be faster than plan A.

- **Plan A:** π A.d, B.d (on the fly) ← σ B.c < 2000 (on the fly) ← ⋈ A.b = B.b (**index nested loop**). Outer input: σ a > 10 (A) using a **B+ tree index on A.a**. Inner: B through a **B+ tree index on B.b**.
- **Plan B:** π A.d, B.d (on the fly) ← ⋈ A.b = B.b (**hash join**). Inputs: σ a > 10 (A) by **file scan**, and σ B.c < 2000 (B) by **file scan**.

Reason 1: the range index scan on A.a can cost more than a file scan. `a > 10` is a **range** that may match a large part of A. If the B+ tree on A.a is **unclustered**, each matching row needs a separate **random page read** (often re-reading the same pages). A **file scan** reads every page of A **once, sequentially**. When many rows match, the sequential scan is cheaper (Tutorial 4: above roughly 10% selectivity a full scan wins).

Reason 2: an index nested loop join probes the B.b index once per outer row; a hash join reads each input about once. For **every** matching A row, plan A walks down the B+ tree on B.b (2–4 I/Os) and fetches matching B rows by **random I/O**. With many outer rows that is thousands to millions of random reads. A **hash join** builds a hash table on one (filtered) input and probes it with the other, reading each table **about once**, mostly sequentially. It is ideal for large inputs and equality joins.

Reason 3 (also acceptable): where the selection is applied. Plan B applies σ B.c < 2000 **before** the join (predicate pushdown), so fewer B rows are hashed and probed. Plan A applies it **after** the join, so the join handles **all** matching B rows and throws many away afterwards.

(Plan A would win only if a > 10 were very selective and the indexes were clustered.)

2020/21 Q7(a) (20m). Discuss the reasons for poor query performance and possible remedies.

Reason	Remedy
Missing or unsuitable indexes → full table scans	Create indexes on filter and join columns; composite or covering indexes
Non-clustered index used for a big range → many random reads	Clustered index on range-heavy columns; let the optimizer choose a scan
Badly written SQL: SELECT *, functions on indexed columns, unnecessary DISTINCT / ORDER BY, correlated subqueries, redundant joins	**Query refactoring:** select only needed columns, rewrite subqueries as joins, keep conditions index-friendly
Missing join conditions (Cartesian products) or a bad join order	Correct the conditions; heuristic + cost-based optimization
Outdated statistics or skewed data → wrong estimates	Update statistics; histograms; partitioning
Large intermediate results; sorts spilling to disk	Push σ and π down; pipelining; more memory
Disk I/O bottlenecks: fragmentation, scattered blocks, slow disks	Contiguous storage, reorganization, buffering/caching, SSDs, RAID striping
Too many indexes slowing down writes	Drop unused indexes
Locking, long transactions	Shorter transactions, suitable isolation levels
The same query run again and again	Query / result caching
Network cost in distributed systems	Data localization, semijoins, replication

Diagnose with the execution plan (EXPLAIN): look for full scans, expensive joins and sorts.

Questions NOT in the mid scope

Past paper questions on Object-Relational databases, Data Warehousing/OLAP, the three-schema architecture and NoSQL are left out, because the mid covers Lessons 1–5 only.